

Correctness Proofs: Distance Calculation, Path Reconstruction, and Generation Computation

1 Recursive Distance Calculation

Source Code

Listing 1: recursive_distance_calculation

```
1 def recursive_distance_calculation(start_node, end_node, neighbor_dict):
2     if start_node == end_node:
3         return 0
4     if neighbor_dict[end_node] == start_node:
5         return 1
6     else:
7         return 1 + recursive_distance_calculation(
8             start_node, neighbor_dict[end_node], neighbor_dict
9         )
```

Correctness of recursive_distance_calculation

Assumption. *BFS works correctly and gives us the best neighbor.*

Base Case. If start and end node are the same, there is no distance between them, return 0. If end node's neighbor is the start node, they have a distance of 1.

Induction Hypothesis. Assume that k is the distance from `neighbor_dict[end_node]` to the `start_node`, want to prove $k + 1$ is the distance to end node.

Explanation and Conclusion. We assume that the distance from any node excluding the end node to `start_node` is k . Note that we are in case 3, where our end node and start node are not identical and that our end node's immediate neighbor is not the `start_node`. In this case, we do the following:

`1 + recursive_distance(start_node, neighbor_dict[end_node], neighbor_dict).`

We add 1 and we proceed to move to the end node's immediate neighbor. By our induction hypothesis, we know that k is the distance from `neighbor_dict[end_node]` to the `start_node`. Since `neighbor_dict[end_node]` to the `start_node` is k and we added 1 for the first call:

`1 + recursive_distance(start_node, neighbor_dict[end_node], neighbor_dict),`

we prove that $k + 1$ is the distance to the end node. Note that this process is guaranteed to terminate since we applied BFS, meaning that as we follow down the neighbors of the end node, we are guaranteed to hit the start node, terminating our function.

2 Path Reconstruction (get_path)

Source Code

Listing 2: get_path

```
1 def get_path(start_node, end_node, neighbor_dict):
2     path_look_up = []
3     while True:
4         if end_node is None:
5             break
6         path_look_up.append(end_node)
7         end_node = neighbor_dict[end_node]
8     path_look_up.reverse()
9     return path_look_up
```

Correctness of get_path

Assumption. *BFS works correctly and gives us the best neighbor.*

Invariant. At any given point, `path_look_up` contains the best path from `target` to the current `end_node`.

Initialization. `path_look_up` is empty, the loop has not started, so the invariant is trivially true.

Maintenance (ignoring break). During the loop, we append `end_node`, which we know has to be part of the best path from `target` to the current `end_node`. We then assign `end_node` to its predecessor going into the next iteration. By appending `end_node` to `path_look_up` and reassigning `end_node` to its predecessor for the next iteration, we still maintain the property that `path_look_up` contains the best path from `target` to the current `end_node`.

Termination. When `end_node` is `None`, this means that we have reached the predecessor of `start_node`, since this BFS implementation returns `None` for `start_node`'s predecessor, and thus we break out of the loop. In the prior iteration, we had appended `start_node`, and thus by breaking out of the loop, we maintain our invariant.

3 Generation Computation (compute_generation)

Source Code

Listing 3: compute_generation

```
1 def compute_generation(person_id, tree):
2     person = tree[person_id]
3     if (not person['parents']) and (not person['spouse']):
4         return 0
5     elif (not person['parents']) and all(
6         not tree[s]['parents'] for s in person['spouse']
7     ):
8         return 0
9     elif person['parents']:
10        return 1 + max(
11            compute_generation(p, tree) for p in person['parents']
12        )
13    elif person['spouse']:
14        return max(
15            compute_generation(s, tree) for s in person['spouse']
16        )
17    return 0
```

Correctness of compute_generation

Assumption. The *parents* relation forms a DAG. No person is their own ancestor, which prevents infinite recursion in the parent direction.

Assumption. One spouse per person. With one spouse, if *A* has spouse *B* and *A* has no parents but *B* does, then *B* walks up their own parents and returns, done. If *B* has no parents, then *A* passes case 2 and we return 0. This prevents the case where *A* fails case 2 because of a parented spouse, and one of *A*'s other spouses has no parents, so we call that spouse and they call *A* back — an infinite bounce in the spouse direction.

Base Case. If our person has no parents and no spouse, they are the root, return 0. If our person has no parents and their spouse has no parents, they are also a root, return 0.

Induction Hypothesis. Assume that k is the deepest number of generations to the target person's parent, want to prove $k + 1$ is the number of generations to the target person.

Explanation and Conclusion. We assume that k is the deepest number of generations to the target person's parent, and want to prove $k + 1$ is the number of generations to the target person. Note that we are in case 3, where our person has parents and we are adding one onto the deepest number of generations returned by the parents:

$$1 + \max(\text{compute_generation}(p, \text{tree}) \text{ for } p \in \text{person}['\text{parents}']).$$

We add 1 and then proceed to move to the parents. By our induction hypothesis, we know that k is the deepest number of generations to the target person's parent, and since we added 1 to the first call, we prove that $k + 1$ is the deepest number of generations to the target person.

Case 4 is likewise the same, but if our target person does not have parents while their spouse does, we use what we did in case 3 — proven correct by the induction hypothesis — to get the spouse’s parents, and return that value for the target person.

This process is guaranteed to terminate. Since the **parents** relation forms a DAG, following parents upward is guaranteed to hit a root, at which point the base case returns 0 and the recursion unwinds. The spouse step of case 4 happens at most once before delegating to the parent recursion of case 3, so it cannot extend the recursion indefinitely.

4 Dynamic Programming: Memoized Generation Computation

Source Code

Listing 4: compute_generation_memo

```
1 def compute_generation_memo(person_id, tree, memo):
2     if person_id in memo:
3         return memo[person_id]
4     person = tree[person_id]
5     if (not person['parents']) and (not person['spouse']):
6         memo[person_id] = 0
7     elif (not person['parents']) and all(
8         not tree[s]['parents'] for s in person['spouse']
9     ):
10        memo[person_id] = 0
11    elif person['parents']:
12        memo[person_id] = 1 + max(
13            compute_generation_memo(p, tree, memo) for p in person['parents']
14        )
15    elif person['spouse']:
16        memo[person_id] = max(
17            compute_generation_memo(s, tree, memo) for s in person['spouse']
18        )
19    else:
20        memo[person_id] = 0
21    return memo[person_id]
```

Why Memoization Applies

Memoization applies because siblings share the same ancestors, and similarly so do cousins. As we can see, there are overlapping recursive calls which can become very expensive in a tree, so dynamic programming can keep us from recomputing.